Reproducing a figure and table from a research paper

Programming and Tools for AI – Week 9

James McDermott







Part 1: a research paper

The problem

Problem. Here is a page from a paper. There is a table of results and a figure with error bars.

This week we want to learn how to produce the table and figure.

metric	setting k	cats off, norm off	cats off, norm on	cats on, norm off	cats on, norm on
chebyshev	1	0.728	0.876	0.728	0.879
	3	0.715	0.896	0.715	0.880
	5	0.722	0.905	0.722	0.893
	9	0.745	0.903	0.745	0.901
	15	0.741	0.899	0.741	0.895
	25	0.744	0.890	0.744	0.826
	45	0.758	0.856	0.758	0.704
euclidean	1	0.732	0.865	0.733	0.871
	3	0.745	0.897	0.745	0.890
	5	0.762	0.918	0.762	0.904
	9	0.756	0.905	0.756	0.901
	15	0.755	0.892	0.755	0.900
	25	0.757	0.898	0.757	0.850
	45	0.765	0.877	0.765	0.822
manhattan	1	0.755	0.877	0.763	0.877
	3	0.754	0.889	0.761	0.887
	5	0.764	0.896	0.762	0.895
	9	0.757	0.907	0.758	0.892
	15	0.760	0.895	0.759	0.893
	25	0.760	0.889	0.759	0.850
	45	0.765	0.877	0.765	0.820

Table 2: Accuracy of our nearest-neighbour classifier at every combination of the four factors, averaged over 25 train/test splits. Normalisation is worth roughly twelve points; the three metrics are indistinguishable; accuracy peaks around $k = 5$ and falls away on either side.

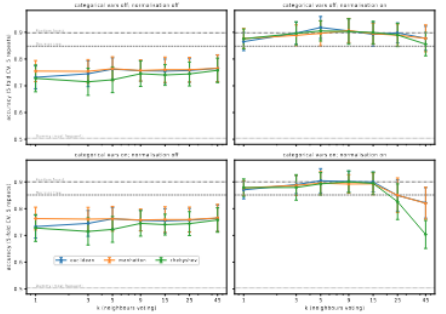


Figure 1: Accuracy against the number of voting neighbours k, for each combination of feature set and normalisation, with one line per distance metric. Error bars are one standard deviation across the 25 splits; horizontal lines mark the reference models of Table 1. Note that the categorical columns change nothing while the features are unnormalised — a zero-or-one column contributes almost nothing to a distance dominated by body mass in grams — and start doing harm as soon as they are given equal weight.

What a results table actually is

A results table is the direct output of a program implementing an experiment:

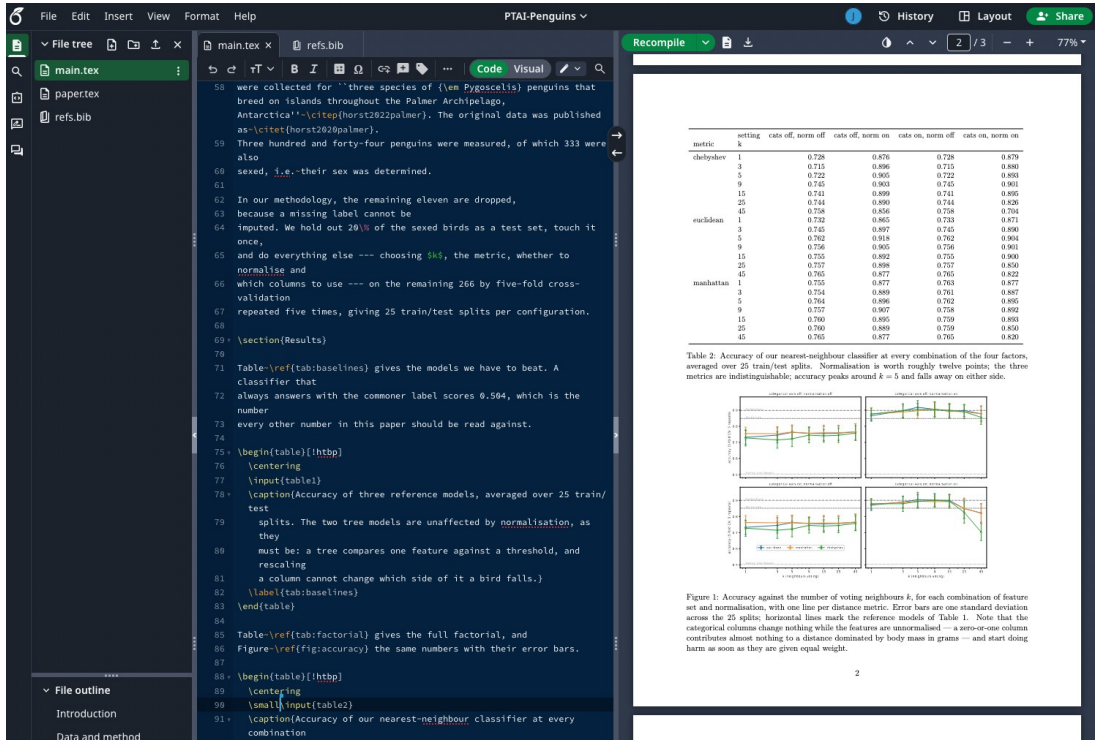
- the rows and columns are the **things we vary**
- the cells are a **summary** — usually a mean — of several runs
- sometimes with a plus-or-minus which is a **spread** over those runs
- one of the rows is usually a **baseline**, ie a simple method

Our template

Here is a LaTeX document on Overleaf. Scientists usually write in LaTeX, not Word. The experimental results are imported to LaTeX from three files:

- `table1.tex`
- `table2.tex`
- `accuracy.pdf`

Our program writes those three files. If we fix a bug and re-run, we just upload the new versions.



Reproducibility is a virtue and a convenience

Manually copy-pasting numbers from the terminal is slow and error-prone, and a good programmer is lazy.

We want automation and reproducibility, eg for these scenarios:

- A reviewer asks us to add one more setting.
- We find a bug in the data loading.
- Six months later we cannot remember which version of the hyperparameters produced Table 2.

Research question

Gorman and colleagues measured 344 penguins at Palmer Station, Antarctica, to study **sexual dimorphism**: whether males and females differ in shape.

We ask their question:

> Given measurements of the penguin's bill, flipper and body mass, can we tell a male penguin from a female one?

This is a **classification** problem. The thing we predict is one of a fixed set of labels — here just two, `Male` and `Female`.

The data

`data/penguins.csv`, 344 rows and 7 columns.

species	island	bill_ln	bill_dp	flip_ln	mass	sex
Adelie	Torger	39.1	18.7	181.0	3750.0	Male
Adelie	Torger	39.5	17.4	186.0	3800.0	Female
Adelie	Torger	40.3	18.0	195.0	3250.0	Female
Adelie	Torger	NaN	NaN	NaN	NaN	NaN

- four **measurements**, in millimetres and grams
- two **categorical** columns, species and island
- `sex` is what we are trying to predict
- some missing values NaN
- one penguin was never measured at all

Our plan

1. **Read and clean** the table, in pandas.
2. **Write a classifier**, k-nearest-neighbours, to the scikit-learn interface.
3. **Design an experiment**: what to vary, what to hold back, how many repeats.
4. **Run it** and save the raw results.
5. **Aggregate** into a table and a figure.
6. **Upload** and compile the paper.

Part 2: pandas and data preprocessing

Reading the data

```
import pandas as pd

raw = pd.read_csv("data/penguins.csv")
>>> raw.shape
(344, 7)
>>> raw.columns
Index(['species', 'island', 'bill_length_mm', 'bill_depth_mm',
       'flipper_length_mm', 'body_mass_g', 'sex'],
      dtype='object')
```

`read_csv` might be the most-used function in pandas. It reads a csv to a `DataFrame`, the central Pandas data structure. It has a `shape`, like in Numpy.

A DataFrame is like a dict of columns

```
>>> raw["body_mass_g"]           # one column, by name, like a dict
0      3750.0
1      3800.0
2      3250.0
3         NaN
        ...
343    5400.0
Name: body_mass_g, Length: 344, dtype: float64
```

A DataFrame is like a dict of columns

```
>>> type(raw["body_mass_g"])
<class 'pandas.core.series.Series'>
```

- a **Series** is one column: a numpy array with labels attached
- a **DataFrame** is a dict of Series, all sharing one index
- every column has one `dtype`; different columns may differ

Programmatic Excel: selecting

Anything you would do by clicking in a spreadsheet, instead do it by coding for reproducibility, eg:

```
>>> raw[["species", "body_mass_g"]] # several columns
>>> raw[raw["body_mass_g"] > 5000]  # filter rows, by a boolean
>>> raw["species"].unique()         # unique values in a column
>>> raw["species"].value_counts()   # counting
```

Summarising a column

```
>>> raw["body_mass_g"].mean()
4201.754385964912
>>> raw["body_mass_g"].std()
801.9545356980954
>>> raw["bill_depth_mm"].mean()
17.15116590643278
>>> raw["bill_depth_mm"].std()
1.974793156816781 6
```

Notice the standard deviations: **802 grams** and **2 millimetres**. They will affect our results a lot, later on.

Missing values

Pandas writes a missing value as NaN, a float meaning “not a number”.

```
>>> raw.isna().sum()
species                0
island                 0
bill_length_mm         2
bill_depth_mm          2
flipper_length_mm      2
body_mass_g            2
sex                   11
dtype: int64
```

NaN is not equal to anything, **not even itself**, so we cannot test with ==. We use `.isna()`.

Two kinds of missing data

A missing **measurement** is a missing input. We often just guess a value using **imputation**.

A missing **label** should not be imputed. We drop those rows.

In this dataset, after dropping those rows, there's no need for imputation.

```
labelled = raw.dropna(subset=["sex"]) # 344 -> 333
labelled.isna().sum().sum()           # 0: no missing vals
```

Here, `.isna()` returns a `DataFrame` containing `bool`s. Then `.sum()` gives a number **per column**. Then `.sum()` sums those.

One-hot encoding aka dummy variables

ML methods can't handle categorical inputs like `"Gentoo"` and `"Adelie"`. **One-hot encoding** replaces the `Species` column with one column per value:

```
>>> pd.get_dummies(labelled[["species"]], columns=["species"],
dtype=int)
     species_Adelie  species_Chinstrap  species_Gentoo
0                 1                  0               0
1                 1                  0               0
2                 1                  0               0
4                 1                  0               0
..              ...                ...             ...
343               0                  0               1
```

Always one 1 in every row. Note the index: penguin 3 was dropped on the last slide.

X and y for Scikit-Learn

```
cols = ["bill_length_mm", "bill_depth_mm",
        "flipper_length_mm", "body_mass_g",
        "species", "island"]
# a table of numbers, one row per penguin
X = pd.get_dummies(labelled[cols], dtype=float)
# one label per row
y = labelled["sex"].to_numpy()
```

Then X and y are the shape scikit-learn expects:

- X: rows are **samples**, columns are **features**
- y: one entry per row of X

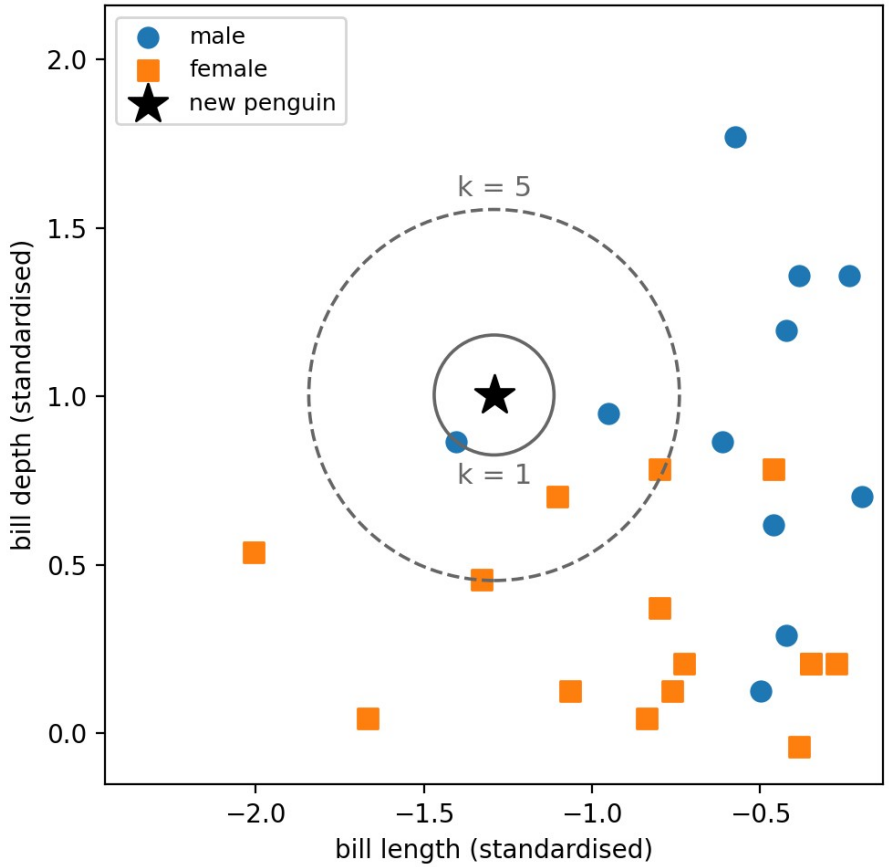
Part 3: classification with k-NN and scikit-learn

k-nearest neighbours

To label a new penguin:

1. measure the Euclidean distance to every penguin already seen
2. take the **k** closest ones
3. they **vote** on label

There is no training per se — “fitting” is just remembering the data.



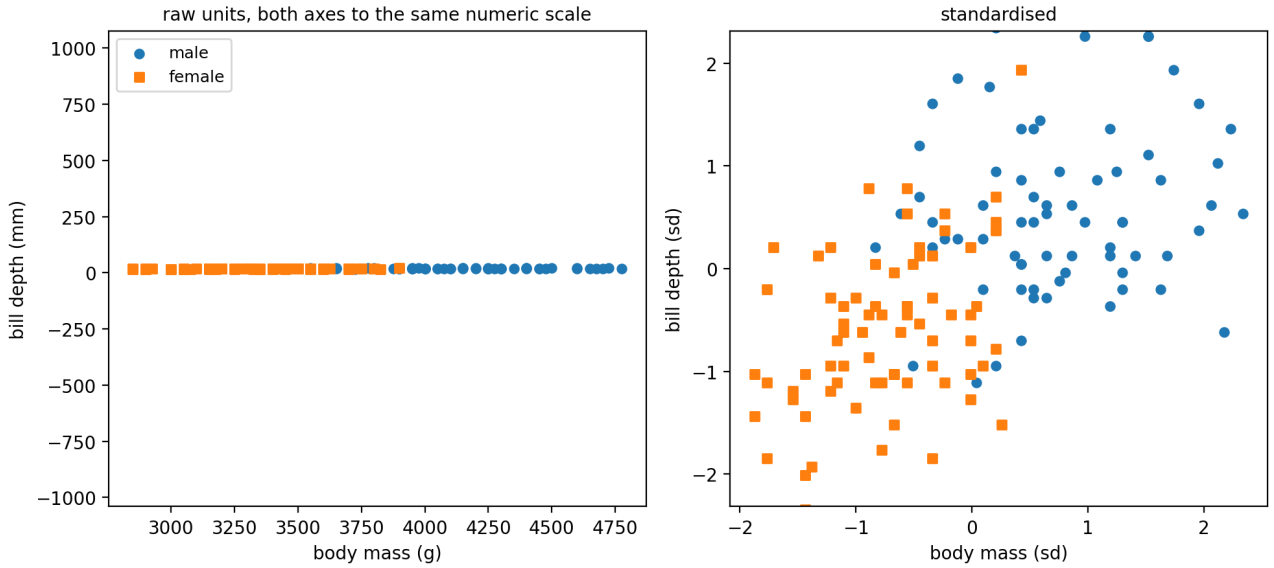
k changes the answer

- **k = 1**: the prediction is set by the closest single point. Sensitive to an unusual sample.
- **k = 45**: forty-five birds vote, so local detail is smoothed away.
- **k = everything**: every query gets the same answer — the commonest label.

A **common pattern**: the hyperparameter **k** can be too low or too high, and the best value is somewhere in the middle.

Distance needs a common scale

Recall: two of our features have standard deviations of **802** and **2**. The mass (which has high std dev) drowns out other features. `StandardScaler` fixes it: subtract each column's mean, divide by its standard deviation, and every column has mean 0 and spread 1.



The same penguins, before and after normalisation. On the left, distance is body mass and nothing else; on the right, every column counts equally.

KNN class: `__init__` and `fit`...

Just like in `SimpleDict`, we use OOP to wrap our code in a nice interface. Scikit-Learn provides `BaseEstimator` and `ClassifierMixin` classes which we **inherit from**:

```
class KNN(BaseEstimator, ClassifierMixin): # inherit

    def __init__(self, k=5, metric="euclidean"):
        self.k = k
        self.metric = metric

    def fit(self, X, y):
        self.X_ = np.asarray(X, dtype=float)
        self.y_ = np.asarray(y)
        self.classes_ = np.unique(y)
        return self
```

In other words, our KNN is a **subclass** of `BaseEstimator` and of `ClassifierMixin`. A `mixin` is a type of class designed to be inherited from like this.

… and predict

The distance logic here is familiar from our photo histograms.

```
    def predict(self, X):
        distances = cdist(np.asarray(X, dtype=float), self.X_,
                          metric=self.metric)
        nearest = np.argsort(distances, axis=1)[:, :self.k]
        return np.array([Counter(self.y_[row]).most_common(1)[0][0]
                         for row in nearest])
```

Recall `cdist` calculates all-to-all distances. Here: every query row against every training row. `argsort` puts the nearest first. The nearest k become the voters.

Recall `Counter.most_common(1)` gives eg: `[('female', 26)]` so this will return `np.array(['female'])`.

Four conventions and their motivations

1. **`__init__` stores its arguments and does nothing else.** Other parts of Scikit-Learn including `clone` and `GridSearchCV` work by reading the `__init__` signature and expecting to find `self.k`.
2. **Anything learned gets an underscore**: X_, not X. Thus `fit` marks its outputs.
3. **`fit` returns `self`**, so `est.fit(X, y).predict(X)` works.
4. **Inherit from `BaseEstimator` and `ClassifierMixin`** and we get `get_params`, `set_params` and `score` for free. All classifiers can use the same `score` function.

instantiate, then fit, predict, score

```
>>> model = KNN(k=5) # create an instance of class KNN
>>> model.fit(X_train, y_train)
KNN()
>>> model.predict(X_test)[:4]
array(['Female', 'Male', 'Female', 'Female'], dtype='<U6')
>>> model.score(X_test, y_test) # accuracy: higher is better
0.746268656716418
```

Every scikit-learn classifier has these three methods.

Duck typing again

```
for est in [KNN(k=5), KNeighborsClassifier(5),
            DecisionTreeClassifier(), DummyClassifier()]:
    print(cross_val_score(est, X, y, cv=cv).mean())
```

`cross_val_score` never asks what any of these objects **is**.

`cross_val_score` is **polymorphic** because it works with different types, as long as they provide `fit`, `predict`, and `score` when it calls them.

The baseline

`DummyClassifier(strategy="most_frequent")` predicts whichever label was most common in training. It does not look at the individual penguin.

```
Dummy (most frequent)   0.5038
```

A baseline like this establishes a point of comparison for other methods.

Part 4: experimental protocol

Factors: things to investigate

factor	levels
normalisation	off, on
categorical features	off, on
metric	euclidean, manhattan, chebyshev
k	1, 3, 5, 9, 15, 25, 45

We have two types of factors: normalisation and categorical features are about data representation. Metric and k are about the model.

Model hyperparameters versus data representation

Model hyperparameters like k belong in the model, ie set in `KNN.__init__`.

Data representation choices like “should we include categorical features” are independent of the model. They should be set in the experiment loop, not inside the model.

One at a time is not enough

A tempting approach is one-factor-at-a-time (OFAT): fix all hyperparameters, vary k, pick the best k; then fix k, vary the metric; and so on.

OFAT misses **interactions**, eg the best k when unnormalised is different from the best k when normalised.

The alternative is **full factorial**: run every combination. 2 x 2 x 3 x 7 = **84** cells.

itertools.product gives a factorial design

```
NORMS = [False, True]
KS = [1, 3, 5, 9, 15, 25, 45]
CATEGORICALS = [False, True] # should we include cat features?
METRICS = {"euclidean": "euclidean",
           "manhattan": "cityblock", # we need both later
           "chebyshev": "chebyshev"}
for cats, norm, metric, k in itertools.product(CATEGORICALS, NORMS,
                                               METRICS, KS):
    ...
>>> len(list(itertools.product(CATEGORICALS, NORMS, METRICS, KS)))
84
```

Hold out data at the start

```
dev, test = train_test_split(rows, test_size=0.2, stratify=y,
                             random_state=SEED)
```

- the test set is **put away** and touched exactly once, at the end
- `stratify=y` keeps the male/female balance the same in both halves
- all our work in choosing k, the metric, whether to normalise etc uses the development set only, not the test set.

Different train-test splits give varied results

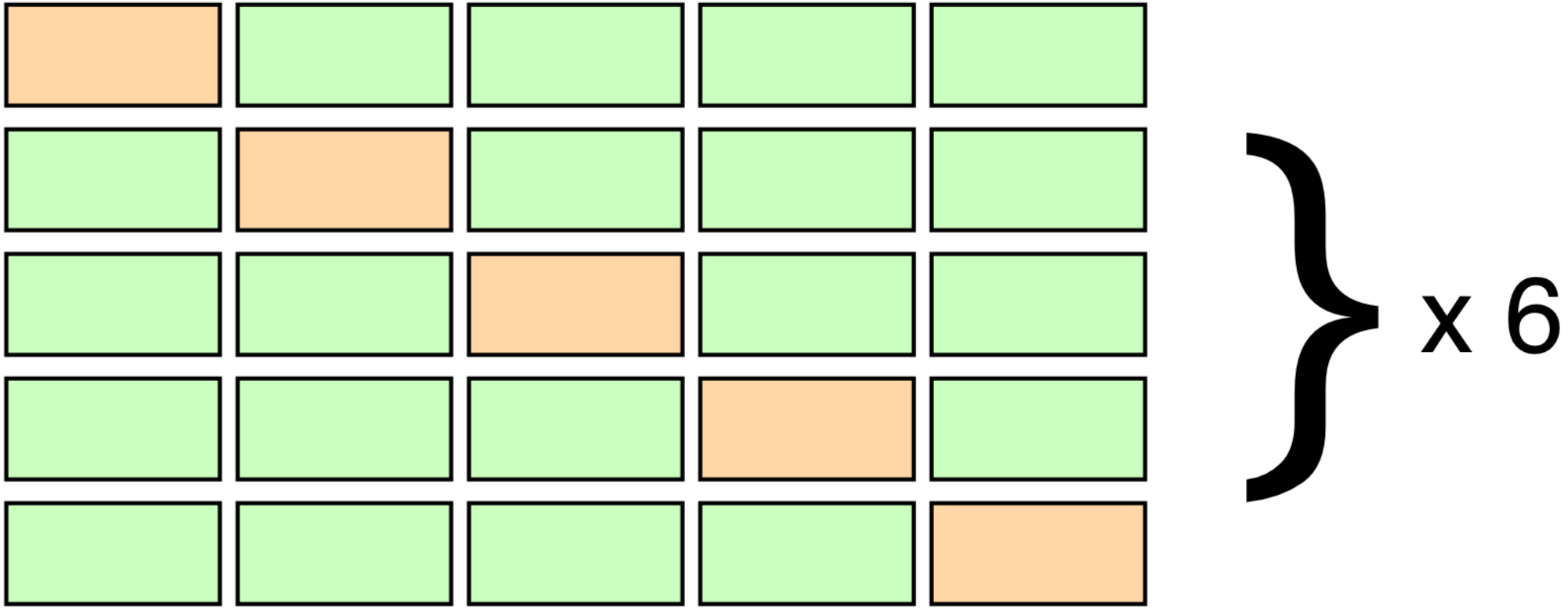
When using the same classifier and the same data, but 10 different train-test splits, we get different accuracy values:

```
0.9104 0.8806 0.9701 0.8657 0.9254 0.8955 0.9254 0.9552 0.9254
0.9701
```

We see a large spread of **10 percentage points**.

A single number has no error bar. It is not a result; it is an anecdote.

Cross-validation: 5x CV with 6x repeats



5x CV with 6x repeats. `RepeatedStratifiedKFold(n_splits=5, n_repeats=6)` gives 30 scores in total. We shuffle and cut the data into 5 folds; use 4 for train and 1 for eval; then rotate so each 1 is used; then reshuffle and go again for 6 repeats.

The protocol function prevents unwritten knowledge

Unwritten knowledge is bad because we'll forget! Or our colleagues won't know what we did. Or other scientists. Or ourselves, 6 months later.

So the protocol function sets up the train-test split and the CV.

```
def protocol(y):
    rows = np.arange(len(y))
    dev, test = train_test_split(rows, test_size=0.2, stratify=y,
                                 random_state=SEED)
    cv = RepeatedStratifiedKFold(n_splits=SPLITS,
                                 n_repeats=REPEATS,
                                 random_state=SEED)
    return dev, test, cv
```

Data leakage

We already know the principle: keep test data totally unseen, else it is “cheating”.

`StandardScaler` learns a mean and a standard deviation. It’s important that preprocessing learning like this should be done on training data alone.

A Scikit-Learn **pipeline** can put together preprocessing and learning components into a single object which provides the same API as an Estimator (fit, predict, score). Using a pipeline here ensures the `StandardScaler` runs on the training data only.

```
make_pipeline(StandardScaler(), KNN(k=k))
```

Tidy results

```
model cats  norm met k setting           seed splits reps spl  acc
KNN   False True euc 5 cats off, norm on    0      5    6   0 0.92
KNN   False True euc 5 cats off, norm on    0      5    6   1 0.94
...
```

The experiment writes 2880 rows to `results.csv`. It is in **tidy format**: one row per observation, including all the inputs and outputs of that observation.

Running the grid in parallel

Each run within a 5x6 CV is independent. Work like that is called **embarrassingly parallel**.

```
cross_val_score(estimator, X, y, cv=cv, n_jobs=-1)
```

`n_jobs=-1` means share all the work (5 folds x 6 repeats) within this `cross_val_score` **across every core on your CPU**.

Checkpointing

Each CV's rows are appended to `results.csv` when it finishes running, using `mode="a"`.

```
pd.DataFrame(rows).to_csv(path, mode="a", index=False,
                          header=not path.exists())
```

If it crashes during the run, we re-run and it will **resume**. Our toy experiment takes 16 seconds, so it doesn't really need this, but real ones do.

Resuming is a set difference

An experimental setup is identified by its **factors**. We summarise them as a **key**, read the keys already in the file, and run the ones that are missing:

```
CELL = ["model", "cats", "norm", "metric", "k"]

counts = df.groupby(CELL).size()      # how many rows per cell?
done = {key for key, n in counts.items()
        if n == SPLITS * REPEATS}     # all 30 there: finished
todo = planned_cells() - done
```

Part 5: outputs

From long to wide

`results.csv` is **long**, one row per run, because that shape is easy to save.

Tables in papers are **wide**.

`pivot_table` transforms long to wide.

pivot_table

```
knn.pivot_table(index=["metric", "k"], columns="setting",
values="accuracy")
```

- `index`: what goes down the side
- `columns`: what goes across the top
- `values`: what fills in the table cells
- it automatically calculates the mean.

Name the columns before pivot, not after

We put the column heading in the table as an ordinary column while the runs are happening:

```
def label(cats, norm):
    return (f"cats {'on' if cats else 'off'}, "
            f"norm {'on' if norm else 'off'}")
```

This makes table creation easy later. More sophisticated methods using multi-indexes allow nicer tables but are out of scope for now.

groupby and agg

The figure will need a mean and a std dev. `groupby` splits, `agg` summarises each group to give 84 rows out:

```
df.groupby(["cats", "norm", "metric", "k"])
["accuracy"].agg(["mean", "std"])
```

```
                                mean       std
cats  norm  metric    k
False False chebyshev 1   0.722444  0.048474
                      3   0.708584  0.049158
                      5   0.720510  0.047180
...                           ...       ...
True  True  manhattan 25  0.850116  0.065616
                      45  0.818344  0.055804
```

Writing the table out

```
table.to_csv("table1.csv", float_format="%.3f")
table.to_latex("table1.tex", float_format="%.3f", multirow=False)
```

`to_latex` writes a table suitable for direct use in LaTeX.

Hello World in matplotlib

```
import matplotlib
matplotlib.use("Agg")      # don't open an MPL window
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.plot([1, 3, 5, 9], [0.87, 0.89, 0.92, 0.90], marker="o")
ax.set_xlabel("k")
ax.set_ylabel("accuracy")
fig.savefig("toy.png", dpi=200)
```

A **figure** is the page. An **axes** is one set of axes drawn on it. Many matplotlib crashes come from confusing them.

A grid of panels

```
fig, axes = plt.subplots(2, 2, figsize=(11, 8), sharex=True,
sharey=True)
axes[0][1].plot(...)
```

We have four factors and two dimensions to draw in. Two of the factors give the **grid of panels**; two go **inside** each panel.

`sharey=True` is useful: without it each panel would have its own scale and the panels would be hard to compare.

Error bars

```
ax.errorbar(ks, means, yerr=sds, marker="o", capsize=3,
label=metric)
```

Each error bar is the standard deviation over 30 runs.

If two lines' error bars strongly overlap, it may not be a significant difference.

Reference lines

```
ax.axhline(0.5038, color="grey", linestyle=":", linewidth=1)
```

We include the baselines as horizontal lines for reference.

Saving: png and pdf

```
for suffix in (".png", ".pdf"):
    fig.savefig(path.with_suffix(suffix), dpi=200)
```

- **png** is pixels. Fine on a slide or a web page; it blurs if enlarged.
- **pdf** is vector graphics, suitable for use in LaTeX.

Matplotlib gallery



Don’t write matplotlib from memory. Instead find a plot in the gallery that looks like what we want, and start from there.

Reading the result

```
norm   off 0.7499   on 0.8795
metric cheb 0.8039  eucl 0.8191  manh 0.8210
k      1 0.8069  5 0.8256  9 0.8273  45 0.7944
cats   off 0.8202  on 0.8092
```

- **normalisation** has a large effect.
- **metric** spans 1.7 points against a spread of 4.4, so there is no significant difference.
- **k** peaks in the middle, as expected.
- the categorical columns made it worse…
- … but only when normalisation is switched on
- … ie **the two factors interact.** Without a factorial design we would miss this effect.

Final evaluation on the test set

```
Best on development: cats off, norm on, euclidean, k=5
  cross-validated 0.9179
  held-out test   0.9104
```

The result of the best model (chosen by performance on the dev set) on the test set (hold-out set) is often a bit worse, because we pick the best of 84 setups via the cross-validated score, which is a bias. The test number is “honest”.

Now make the paper

1. change `SEED` and run `python E01_penguins.py`
2. upload `table1.tex`, `table2.tex`, `accuracy.pdf` to your Overleaf copy
3. put your own name on it
4. **read it**, and correct any number quoted in the prose
5. export the pdf

Step 4 is the real work. If you change the seed you may expect some of the numbers to change.

Summary

- A results table and a figure are **program output**. It is crucial to produce them in an automated way.
- Checkpointing and parallel code.
- **pandas** `DataFrame` is a dict of columns: `read_csv`, select, `groupby`, `pivot_table`, and produce csv or LaTeX.
- Writing a classifier with the **scikit-learn API** (`fit`, `predict`, `score` and the convention for trailing-underscore) allows us to easily compare our classifier against built-in ones (polymorphic code thanks to Python duck-typing).
- A **full factorial** design is common. `itertools.product` or `GridSearchCV`.
- **Hold data back, repeat enough to get an error bar, and compare against a baseline.** Without all three we cannot identify real results. `train_test_split`, `cross_val_score`.

Not examinable

This week included a lot of fiddly and technical code, and some ML-specific code, and some plotting / table code that you don't need to reason about, just run it and see the results. The following will not be examinable as code, but the concepts behind them are.

Pandas: `pandas` as `pd`; `DataFrame`, `Series`, `read_csv`, `to_csv`, `to_latex`, `isna`, `dropna`, `fillna`, `get_dummies`, `value_counts`, `unique`, `agg`, `pivot_table`, `idxmax`.

Scikit-learn: `BaseEstimator`, `ClassifierMixin`, `RepeatedStratifiedKFold`, `StandardScaler`, `make_pipeline`, `DummyClassifier`, `KNeighborsClassifier`, `DecisionTreeClassifier`, `RandomForestClassifier`.

Matplotlib: `subplots`, `errorbar`, `axhline`, `savefig`.